

Activity 7 Document

Title of Activity

- Task Management System
- Short description (what the app does)

-The Task Management System helps users create and manage projects and tasks in one dashboard. Users can add projects with names and descriptions, create tasks with deadlines and status like To Do, Pending, Approved, or Overdue, and assign tasks to team members. Users can also view, edit, and delete projects, tasks, and users. This system helps companies stay organized and make sure no work or client is missed..

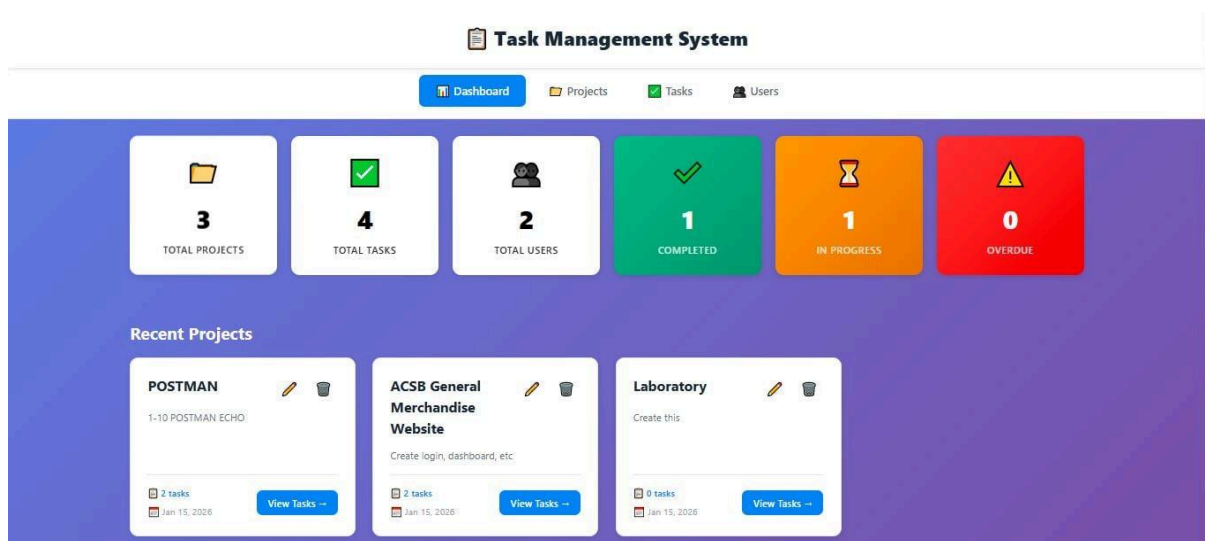
Features:

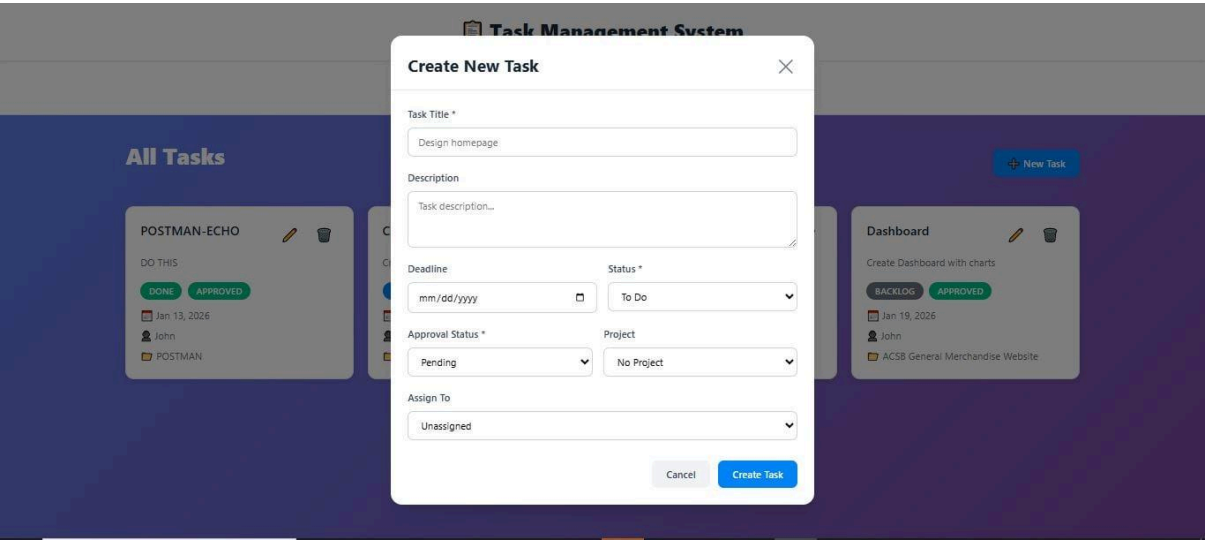
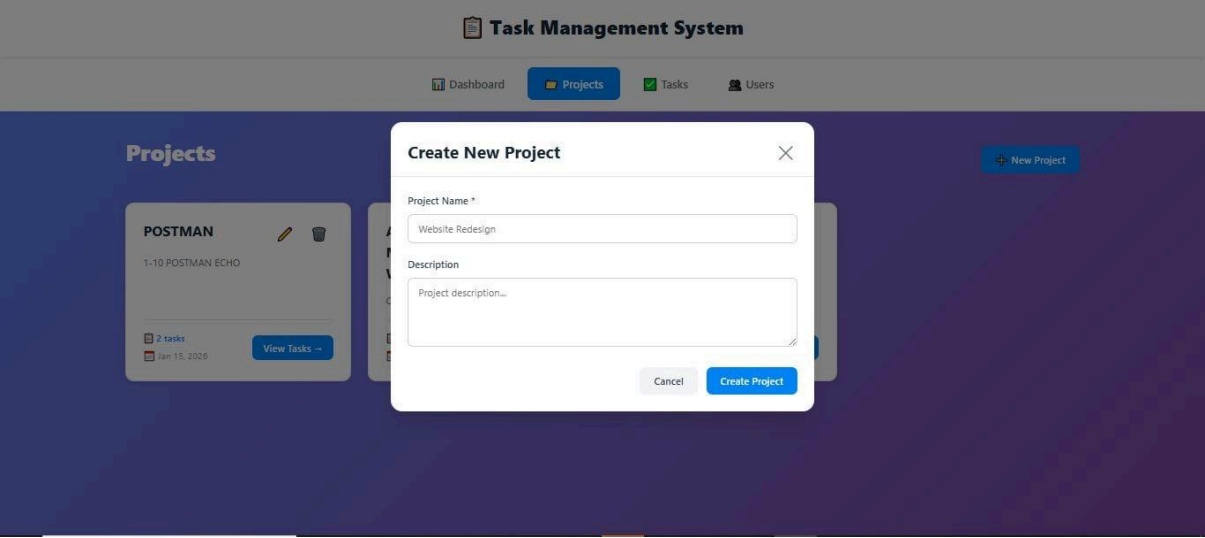
- Dashboard that you can create you can create a project, task, users
 - Dashboard that you can add,edit,update and delete your project , task and save directly to the database and also have a CRUD API that can also be tested in Swagger Docs, Cursor AI and Postman.
 - Postman Echo can be used to test your Postman connection by getting an echo link to test the CRUD API.
- Screenshot(s) of working system (UI + API example)

Working System UI

Frontend /Backend

- **Front End**





Task Management System

Dashboard Projects Tasks Users

Projects

New Project

POSTMAN

1-10 POSTMAN ECHO

2 tasks

Jan 15, 2026

View Tasks --

ACSB General Merchandise Website

Create login, dashboard, etc

2 tasks

Jan 15, 2026

View Tasks --

Laboratory

Create this

0 tasks

Jan 15, 2026

View Tasks --

Task Management System

Dashboard Projects Tasks Users

All Tasks

New Task

POSTMAN-ECHO

DO THIS

DONE APPROVED

Jan 13, 2026

John

POSTMAN

Create Website

Create login, dashboard, etc

TODO APPROVED

Jan 22, 2026

Chloe

POSTMAN

Login

Authentication with JWT Token

IN PROGRESS APPROVED

Jan 17, 2026

Chloe

ACSB General Merchandise Website

Dashboard

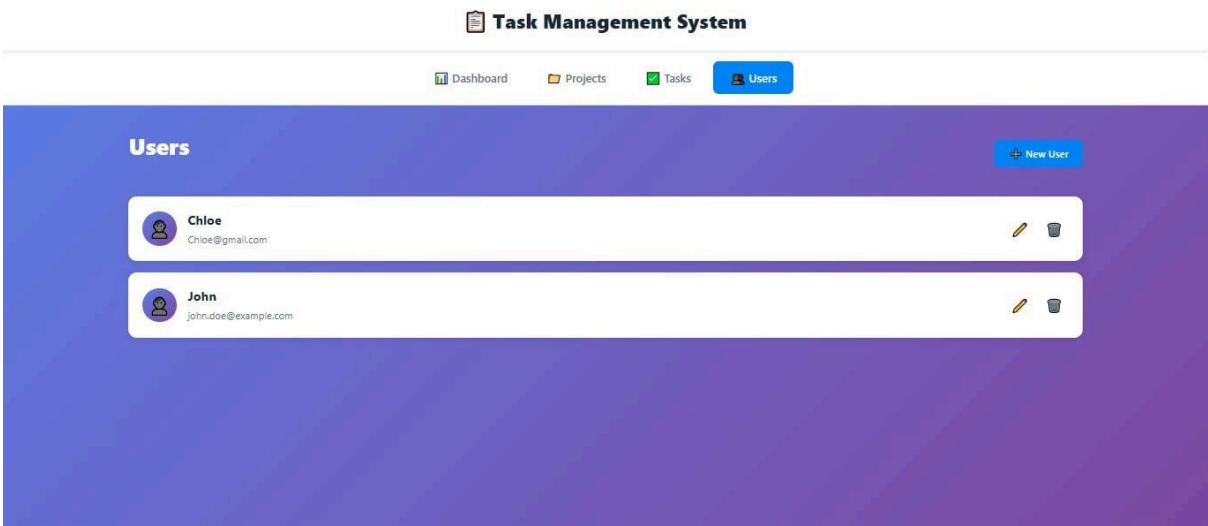
Create Dashboard with charts

BACKLOG APPROVED

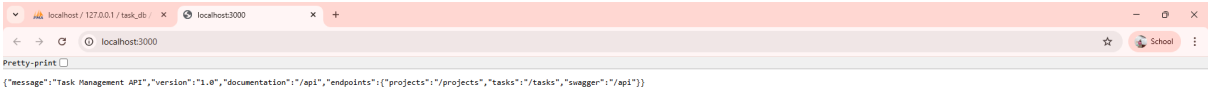
Jan 19, 2026

John

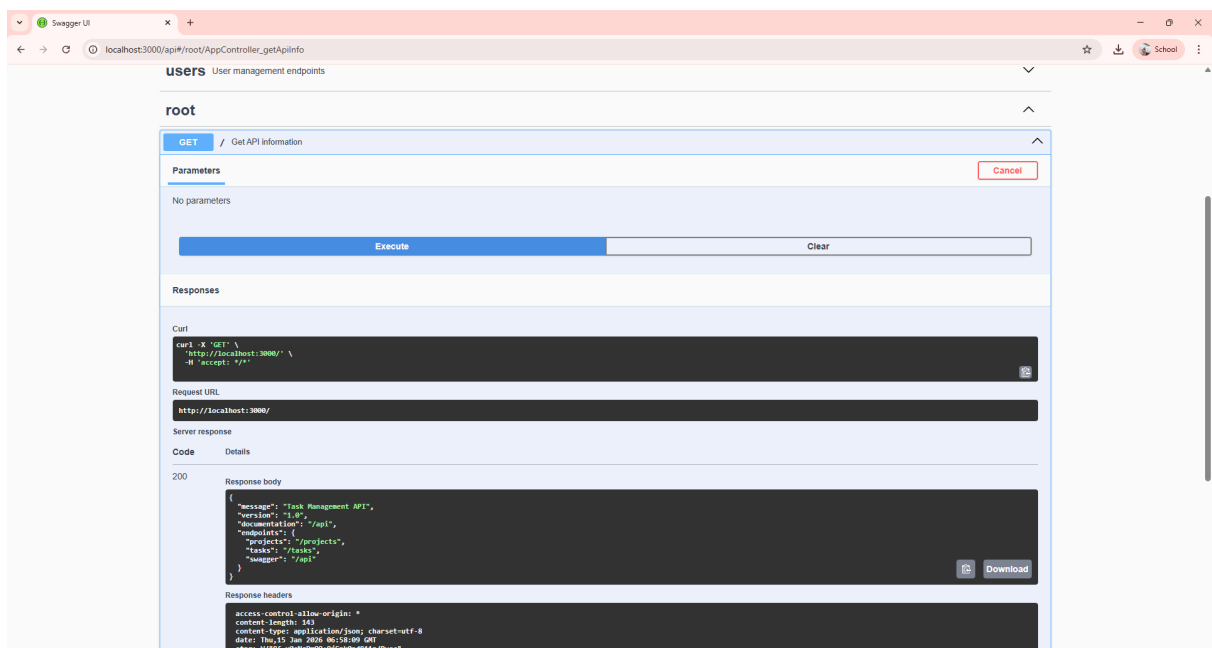
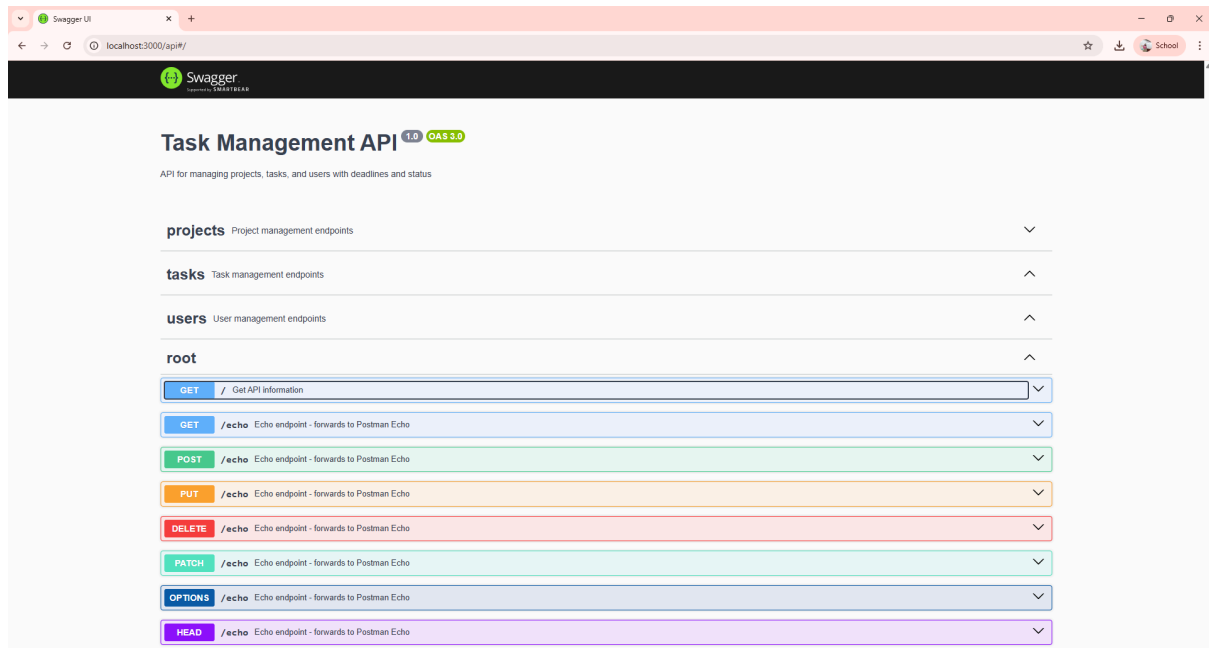
ACSB General Merchandise Website



Backend



API Swagger



Postman Echo

GET HTTP REQUEST/ RESPONSE

The screenshot shows the Postman interface for a GET request. The URL is `[[base_url]]/get/endpoint=tasks`. The query parameters table is as follows:

Key	Value	Description
endpoint	tasks	
Key	Value	Description

The response is a 200 OK status with a JSON body:

```
1 {
2   "args": {
3     "endpoint": "tasks"
4   },
5   "headers": {
6     "host": "postman-echo.com",
7     "accept-encoding": "gzip, br",
8     "accept": "*/"
```

POST HTTP REQUEST/ RESPONSE

The screenshot shows the Postman interface for a POST request. The URL is `[[base_url]]/post`. The body is raw JSON:

```
1 {
2   "name": "Website Redesign",
3   "description": "Redesign the company website",
4   "status": "pending"
5 }
6
```

The response is a 200 OK status with a JSON body:

```
1 {
2   "args": {},
3   "data": {
4     "name": "Website Redesign",
5     "description": "Redesign the company website",
6     "status": "pending"
7   }
8 }
```

PATCH HTTP REQUEST/ RESPONSE

The screenshot shows the Postman interface for a PATCH request. The URL is `[[base_url]]/patch`. The body is raw JSON:

```
1 {
2   "id": 18,
```

DELETE HTTP REQUEST/ RESPONSE

The screenshot displays a REST client interface for "Activity 7". The request is a DELETE method to the endpoint `{{base_url}}/delete`. The request body is a JSON object:

```
{  "id": 58}
```

. The response is a 200 OK status with a response time of 469 ms and a body size of 1.04 KB. The response body is a JSON object:

```
{  "msg": {},  "data": {    "id": 58  },  "files": {},  "form": {},  "headers": {    "Host": "localhost:3000"  }}
```

Activity 7

Save Share

DELETE `{{base_url}}/delete` Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Cookies Beautify

```
1 {
2   "id": 58
3 }
4
```

Body Cookies (3) Headers (12) Test Results

JSON Preview Visualize

```
1 {
2   "msg": {},
3   "data": {
4     "id": 58
5   },
6   "files": {},
7   "form": {},
8   "headers": {
9     "Host": "localhost:3000"
10  }
11 }
```

200 OK - 469 ms - 1.04 KB

Instructions on how to run the project

Activity 7 Requirements

- Node.js (version 16 or higher)
- React (Typescripts & Javascripts)
- npm (Node Package Manager)
- Git (for cloning the repository)

Step-by-Step Instructions

1. Clone the Repository

- Open Visual Studio Code.
- Open the terminal
- Run the following command to clone the GitHub repository:

```
git clone https://github.com/CVSU-IMUS-BSIT-4A/Group-4A-Repository.git
```

- Navigate to the project folder:

```
cd C:\Users\demro\OneDrive\Desktop\Group-4A-Repository-1
```

2. Install Backend Dependencies

- Go to the backend directory:
- cd C:\Users\demro\OneDrive\Desktop\Group-4A-Repository-1\Activity 7\backend

Install all required dependencies:

```
npm install
```

3. Set Up the Database

Make sure XAMPP is running (Apache and MySQL).

Open phpMyAdmin at:

<http://localhost/phpmyadmin>

Create a new database (e.g., task_db).

Once configured properly, the backend will automatically create the necessary tables (for email, password, etc.) in your database.

4. Start the Backend Server

If your package.json includes a start script (e.g., start:dev), run the backend server using

```
npm run start:dev
```

After it starts successfully, the backend should be running at:

<http://localhost:3000>

You can check the API documentation or routes by visiting:

<http://localhost:3000/api>

5. Install Frontend Dependencies

Open a new terminal in Visual Studio Code.

Navigate to the frontend directory:

```
cd C:\Users\demro\OneDrive\Desktop\Group-4A-Repository-1\Activity 7\frontend
```

Install the required dependencies:

```
npm install
```

6. Start the Frontend Server

In the same terminal, run:

```
npm run dev
```

After it starts, open your browser and go to:

<http://localhost:5173>

7. Access the Application

Frontend Application: <http://localhost:5173>

Backend API: <http://localhost:3000>

Swagger or API Docs (if available): <http://localhost:3000/api>